

IAD-006 – APRENDIZADO DE MÁQUINA II

Aula 02 – Métodos Ensemble



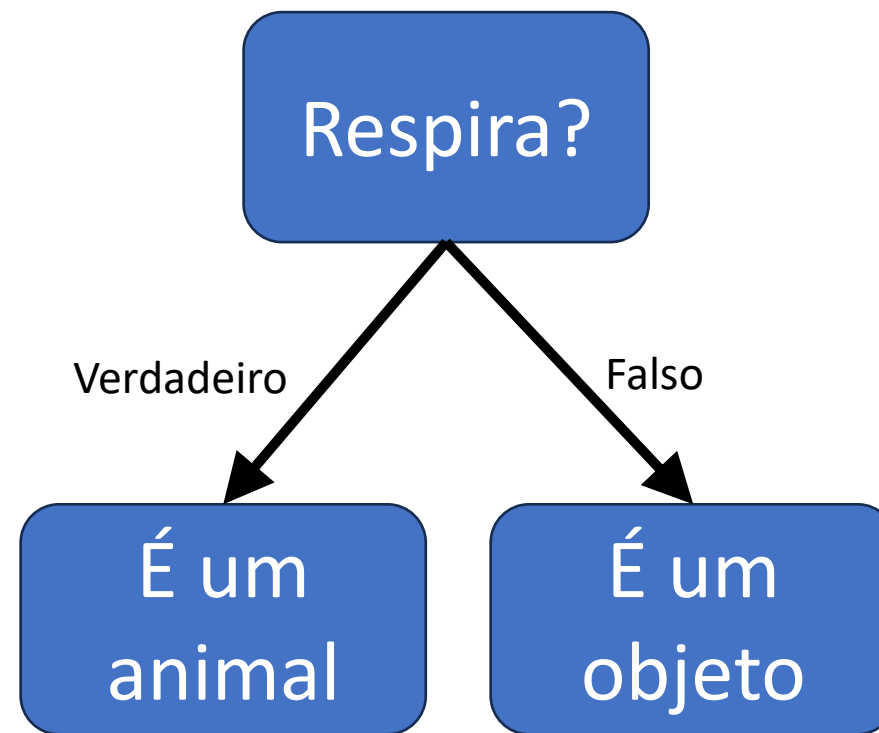
Prof. Dr. Arthur Henrique de Andrade Melani

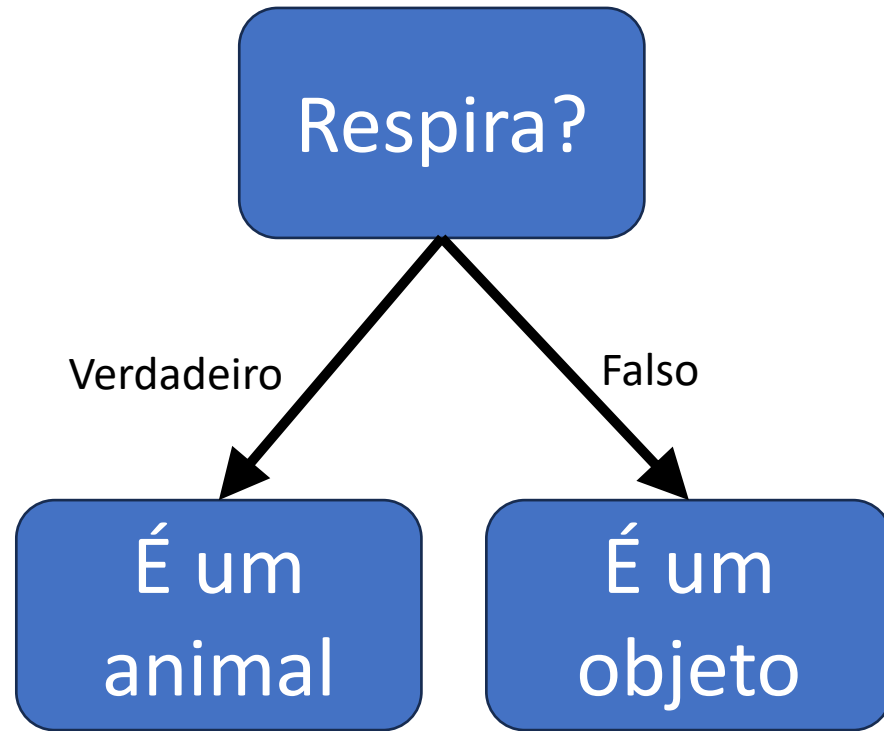
Departamento de Engenharia Mecatrônica e Sistemas Mecânicos

Escola Politécnica da Universidade de São Paulo

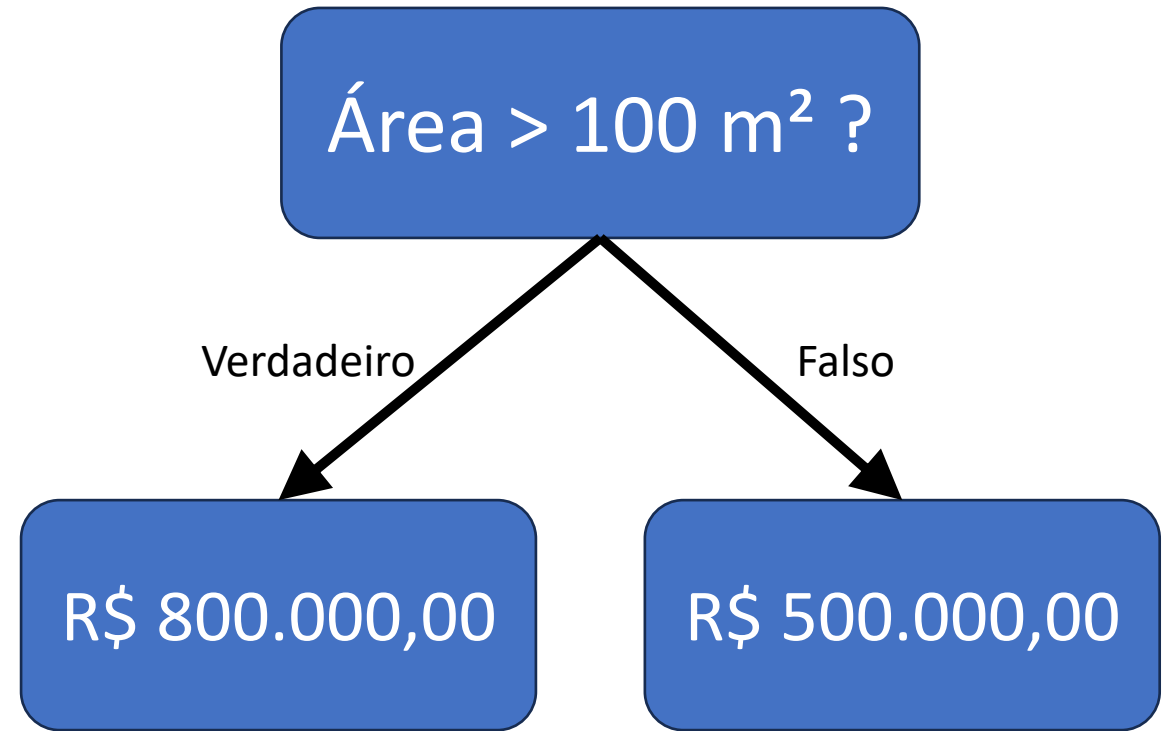
melani@usp.br

- Árvore de Decisão é um modelo de aprendizado **supervisionado** utilizado para **classificação** e **regressão**.
 - Ou seja, podem prever algo categórico ou numérico!
- Representa decisões na forma de uma **estrutura hierárquica** de perguntas e respostas.
- As árvores de decisão aceitam variáveis categóricas e numéricas **sem necessidade de normalização**





- Em **classificação**, cada folha indica uma **classe** (ex: *Mamífero*, *Máquina*).



- Em **regressão**, cada folha indica um **valor numérico estimado** (ex: preço, temperatura).

Limitações de um único modelo de Machine Learning

- Todo modelo tem **viés** e **variância**.
- Um modelo único pode:
 - Ser **muito simples** → alta taxa de erro por subajuste (*underfitting*).
 - Ser **muito complexo** → alta sensibilidade a ruído (*overfitting*).
- Resultados fortemente dependentes de:
 - Divisão dos dados
 - Parâmetros de inicialização
 - Algoritmo escolhido

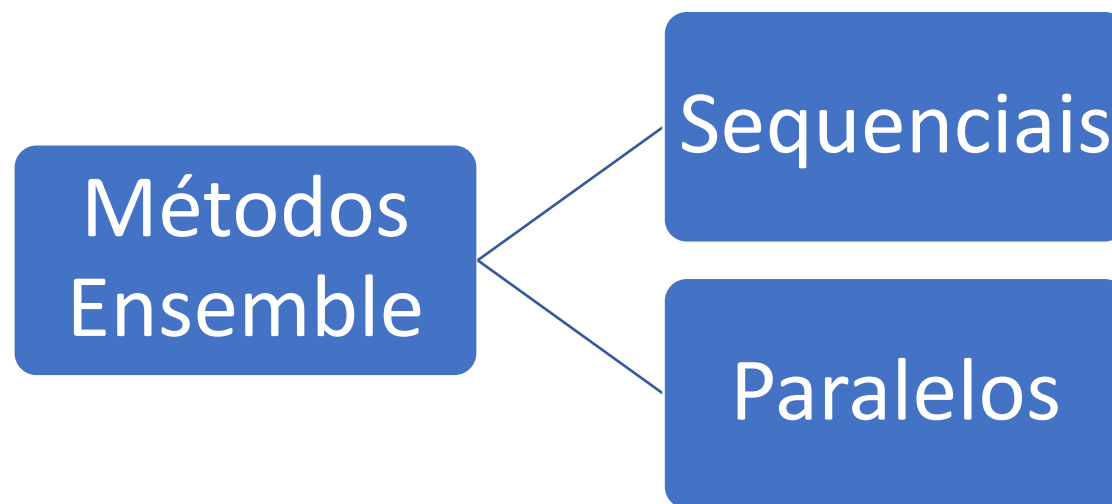
Nenhum modelo é perfeito, mas **vários modelos podem se complementar**

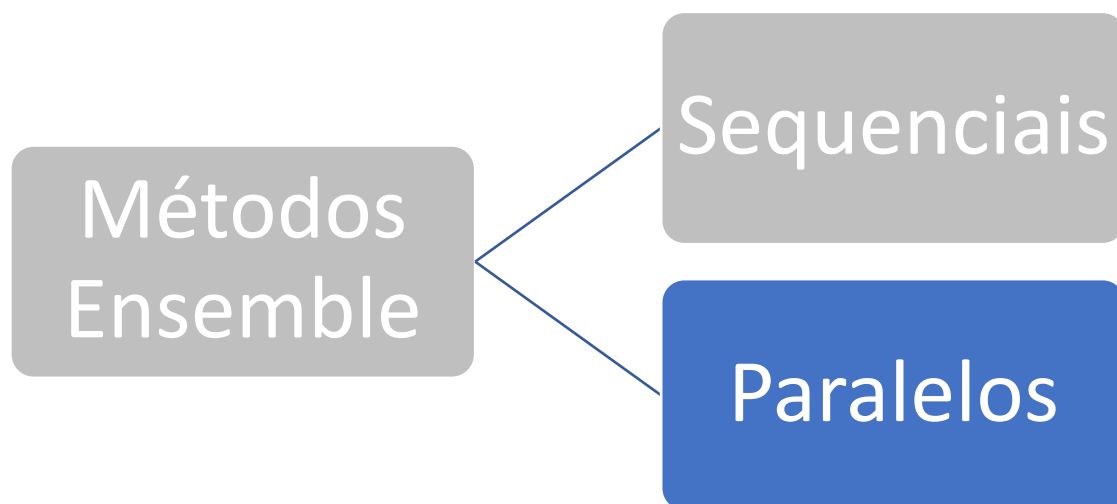
O que são *Ensemble Methods*, ou Métodos de Comitê?

Métodos ensemble são técnicas de aprendizado de máquina que **combinam as previsões de múltiplos modelos base** (também chamados de **estimadores fracos** ou individuais), com o objetivo de construir um **modelo composto mais preciso, estável e generalizável (estimador forte)** do que qualquer modelo individual.



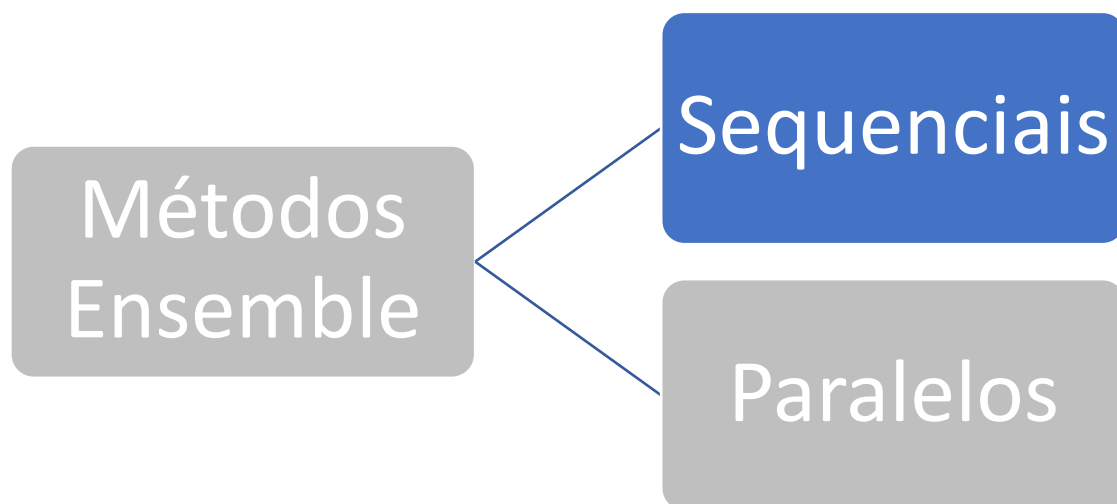
Os métodos ensemble podem ser divididos em dois grupos:





Paralelo

- **Todos os modelos são treinados de forma independente**, em subconjuntos dos dados.
- A ideia é reduzir **variância** combinando modelos que erram de formas diferentes.
- As previsões são **agregadas ao final** (ex.: média, votação).
- **Erro de um modelo não afeta os outros.**
- **Exemplo:** Bagging, Random Forest
- **Aplicação:** Útil quando o modelo base tem alta variância (ex.: árvore de decisão profunda).



Sequencial

- **Os modelos são treinados um após o outro.**
- Cada novo modelo tenta **corrigir os erros do modelo anterior.**
- Foca em reduzir **viés**, aprimorando sucessivamente a predição.
- A combinação final é **ponderada**, dando mais peso a modelos mais precisos.
- **Exemplo:** Boosting, AdaBoost, Gradient Boosting, XGBoost
- **Aplicação:** Útil quando o modelo base tem alto viés (ex.: árvores rasas).

Ensembles Paralelos:

- **Averaging / Voting**
 - Combina previsões de modelos diferentes (ou iguais), por média ou votação.
- **Bagging**
 - Treina vários modelos em subconjuntos aleatórios dos dados (com reposição).
- **Random Forest**
 - Método baseado em bagging com árvores de decisão e seleção aleatória de atributos.

Ensembles Sequenciais:

- **AdaBoost**
 - Modelos fracos treinados sequencialmente, com pesos ajustados para erros.
- **Gradient Boosting**
 - Modelos são adicionados para corrigir os resíduos do modelo anterior, via gradiente.

Averaging / Voting



- *Averaging* e *Voting* são as duas técnicas mais simples de comitê (ensemble)
- São técnicas do tipo Paralela, ou seja, cada modelo é **treinado de forma independente** e as **previsões são agregadas** para obter o resultado final.
- Podem ser aplicadas com **modelos iguais ou diferentes** (SVM, árvore de decisão, rede neural, etc.)

Voting

- Usados para tarefas de classificação
- **Votação majoritária (hard voting):**
 - Cada modelo “vota” na classe.
 - A classe mais votada é a escolhida.
- **Votação na média das probabilidades (soft voting):**
 - Calcula a média das probabilidades preditas por cada modelo. A classe com maior valor médio é escolhida.
 - Requer que os modelos forneçam probabilidades
- **Votação Ponderada**
 - Pode ser aplicado tanto no hard quanto no soft voting.
 - Cada voto tem um peso (ex.: baseado na acurácia do modelo).

Exemplo: classificação de imagens

Modelo	Classe 0 (Gato)	Classe 1 (Cão)	Classe 2 (Pássaro)
Modelo 1	0,4	0,5	0,1
Modelo 2	0,31	0,35	0,34
Modelo 3	0,8	0,1	0,1

- Hard Voting:
 - Modelo 1 → Classe 1 (Cão)
 - Modelo 2 → Classe 1 (Cão)
 - Modelo 3 → Classe 0 (Gato)
 - Resultado por Maioria: Cão
- Soft Voting:
 - Classe 0 (Gato)
 - $(0.40 + 0.30 + 0.80)/3 = 0.50$
 - Classe 1 (Cão)
 - $(0.50 + 0.35 + 0.10)/3 = 0.3167$
 - Classe 2 (Pássaro)
 - $(0.10 + 0.35 + 0.10)/3 = 0.1833$
 - Resultado por Média: Gato

Exemplo: classificação de imagens

Modelo	Classe 0 (Gato)	Classe 1 (Cão)	Classe 2 (Pássaro)
Modelo 1	0,4	0,5	0,1
Modelo 2	0,31	0,35	0,34
Modelo 3	0,8	0,1	0,1

Escolher entre os tipos de votação depende do tipo de saída dos modelos, da confiança relativa entre eles e da natureza do problema.

- **Hard Voting:**
 - Modelo 1 → Classe 1 (Cão)
 - Modelo 2 → Classe 1 (Cão)
 - Modelo 3 → Classe 0 (Gato)
 - Resultado por Maioria: Cão
- **Soft Voting:**
 - Classe 0 (Gato)
 - $(0.40 + 0.30 + 0.80)/3 = 0.50$
 - Classe 1 (Cão)
 - $(0.50 + 0.35 + 0.10)/3 = 0.3167$
 - Classe 2 (Pássaro)
 - $(0.10 + 0.35 + 0.10)/3 = 0.1833$
 - Resultado por Média: Gato

Averaging

- Usado principalmente para tarefas de regressão
- Cada modelo retorna uma saída contínua (por exemplo, valor numérico).
- A saída final é calculada por alguma forma de média:
 - Média simples (uniforme)
 - Média ponderada
 - Mediana, etc.

Weighted Averaging

- A importância de cada preditor pode ser ponderada por uma medida de desempenho de cada modelo.
- Neste caso, cada modelo passa a contribuir de forma diferente para o resultado da predição

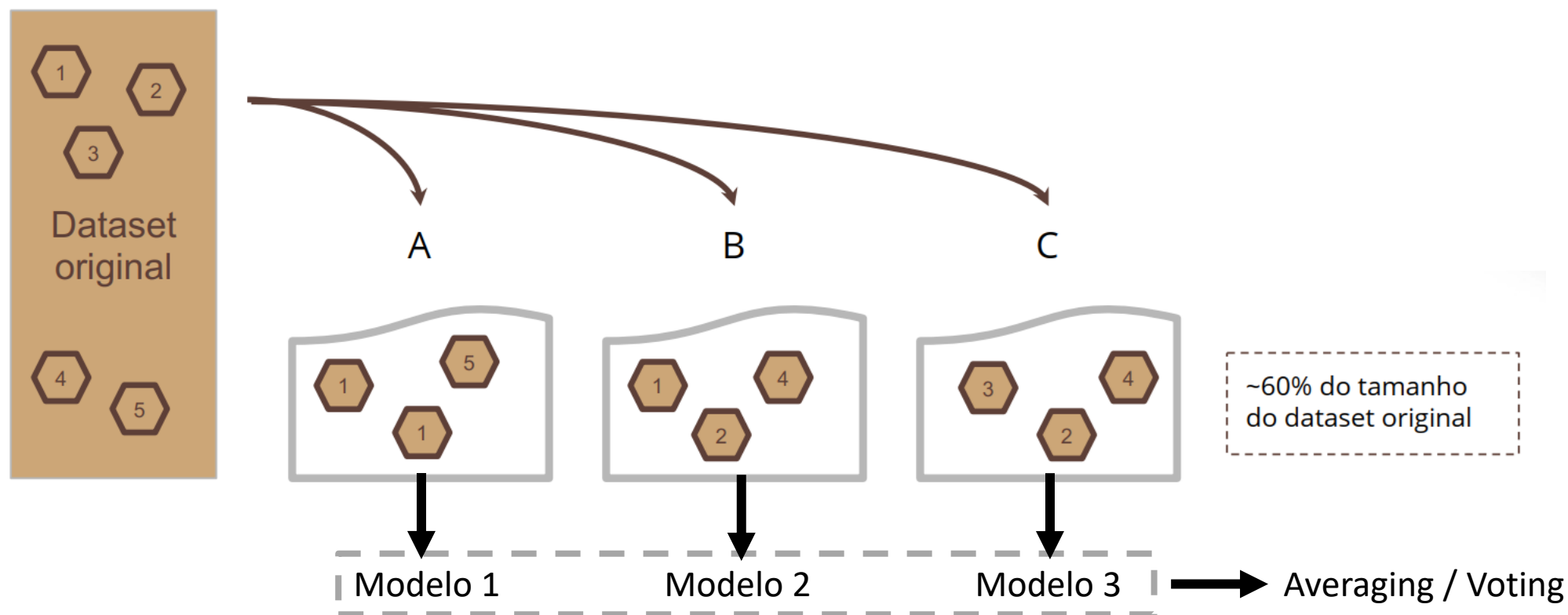
Bootstrap aggregating - Bagging

Bagging é uma técnica de ensemble que visa **reduzir a variância de modelos instáveis**, como árvores de decisão, por meio da **criação de diversos modelos treinados em subconjuntos diferentes dos dados** e da **combinação das suas previsões**.



OBS: Bootstrap aggregating = Agregação por amostragem com reposição

A principal ideia desta abordagem é criar **subconjuntos randômicos dos dados originais**. Uma mesma amostra pode aparecer mais de uma vez em um mesmo subconjunto.



Random Forest

- **Random Forest** é uma técnica baseada em **Bagging**, que combina várias **árvores de decisão** construídas de forma independente, cada uma treinada com:
 - **Subconjuntos aleatórios de dados (bootstrap)**
 - **Subconjuntos aleatórios de features em cada divisão (split)**



Como funciona?

- Para cada árvore:
 - Gerar um **bootstrap sample** do conjunto de treino.
 - Em cada divisão (nó) da árvore, escolher uma **amostra aleatória de atributos** para definir o melhor split.
- Repetir o processo para **várias árvores** (normalmente dezenas ou centenas).
- Combinar as saídas:
 - **Votação (classificação)**
 - **Média (regressão)**

Amostragem aleatória de atributos – o que significa?

- **Na árvore de decisão tradicional:**
 - Durante a construção da árvore, a cada **nó** é necessário escolher o **melhor atributo para dividir os dados**, ou seja, **testar todos os atributos e escolher o que gera menos impureza**
- **No Bagging tradicional:**
 - Cada árvore é treinada com uma **amostra bootstrap dos dados**.
 - Mas em **cada divisão (split)**, a árvore considera **todos os atributos** para escolher o melhor.
- **No Random Forest:**
 - Além da amostragem de dados (bootstrap), o algoritmo adiciona mais aleatoriedade:
 - Em **cada nó da árvore**, é feito um **sorteio aleatório de um subconjunto de atributos**.
 - A divisão é feita **somente entre os atributos sorteados**, não entre todos.

- **Exemplo:**

- Suponha que seus dados tenham **10 features** (A1 a A10).
- Em cada nó, o Random Forest sorteia, por exemplo, **3 features aleatórias** (ex.: A2, A5, A7) e decide o melhor split **apenas entre elas**.
- O número de atributos a serem sorteados por nó é um hiperparâmetro:
 - Para **classificação**, costuma-se usar $\sqrt{d}$ atributos, onde d é o total.
 - Para **regressão**, costuma-se usar $d/3$.

Avaliando uma árvore treinada com *out-of-bag samples*

- No Random Forest, cada árvore é treinada com um subconjunto da base de treino
- Como resultado, cerca de $1/3$ do total da base não é usada na construção de uma árvore
- Essas amostras são chamadas de **Out-of-Bag (OOB)** para essa árvore
- Podemos fazer a predição de todas as amostras que ficaram fora do treinamento e usamos este resultado para avaliar o modelo.

Avaliando o número ideal de features de cada árvore

- É possível melhorar ainda mais o resultado do modelo avaliando qual o número ideal de features de cada subconjunto
- O número de features que tiver o melhor desempenho será usado no modelo final. Podemos pensar aqui na mesma ideia do GridSearch para buscar o melhor hiperparâmetro

A	B	C	D	Label

A	B	C	D	Label

A	B	C	D	Label

Adaboost (Adaptative Boosting)

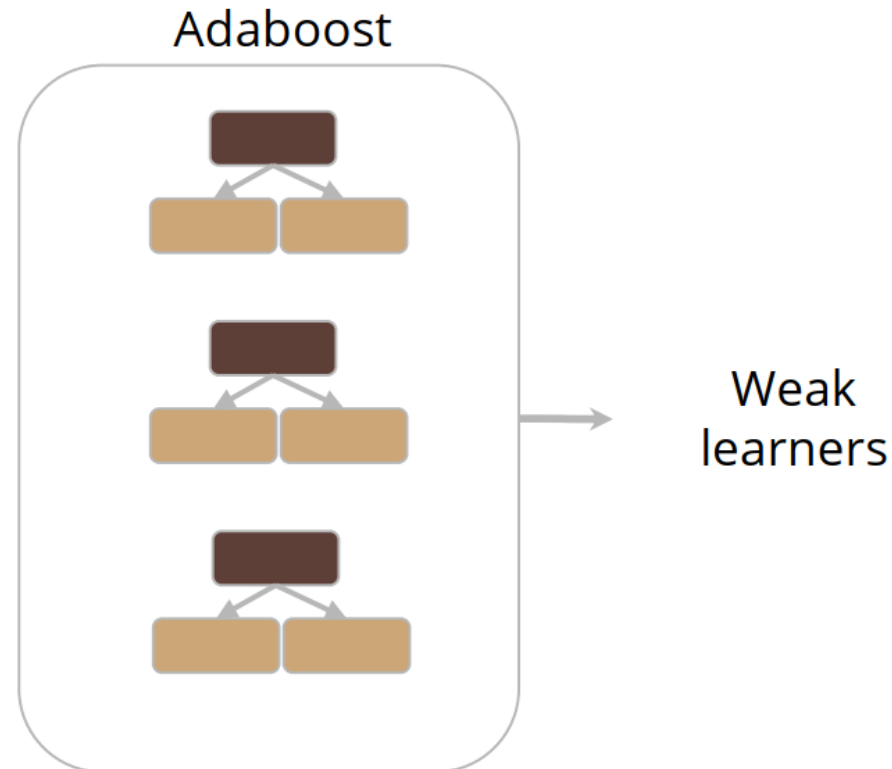
AdaBoost (Adaptive Boosting) é um método de ensemble **sequencial** que combina **múltiplos classificadores fracos** (tipicamente pequenas árvores de decisão) para formar um **classificador forte e preciso**.

- **Como funciona, em essência?**

- Os modelos são treinados **um após o outro**.
- Cada novo modelo é **ajustado para corrigir os erros** cometidos pelos modelos anteriores.
- Durante o treinamento, o algoritmo **aumenta o peso das instâncias/samples mal classificadas**, forçando os próximos modelos a focarem nelas.



- No Adaboost, as árvores são compostas de **um nó raiz e duas folhas**. Essas árvores são chamadas de *stumps* ao invés de árvores de decisão (tocos, em português)
- Cada *stump* é construído com base no erro dos *stumps* anteriores



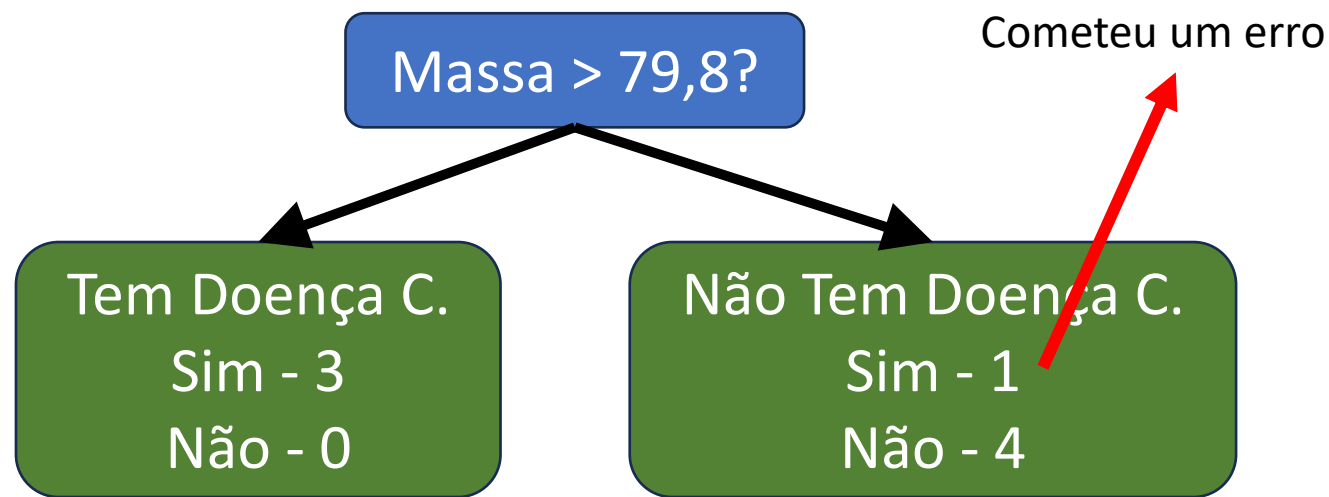
Treinando um Adaboost

- O primeiro stump de decisão é definido da mesma forma que vimos na aula passada, usando as medidas de impurezas (gini, entropy, etc)
- Todas as amostras do dataset começam com **o mesmo peso**. Esta importância de cada amostra **será atualizada** a cada passo a fim de penalizar as amostras que são incorretamente classificadas pelos *stumps* (troncos) anteriores.

Dor no Peito	Artérias Bloqueadas	Massa do Paciente	Doença no Coração	Peso da Amostra
Sim	Sim	93	Sim	1/8
Não	Sim	81	Sim	1/8
Sim	Não	95	Sim	1/8
Sim	Sim	75	Sim	1/8
Não	Sim	70	Não	1/8
Não	Sim	56	Não	1/8
Sim	Sim	76	Não	1/8
Sim	Sim	78	Não	1/8

Treinando um Adaboost

- O primeiro stump de decisão é definido da mesma forma que vimos na aula passada, usando as medidas de impurezas (gini, entropy, etc)
- Todas as amostras do dataset começam com o mesmo peso. Esta importância de cada amostra será atualizada a cada passo a fim de penalizar as amostras que são incorretamente classificadas pelos *stumps* (troncos) anteriores.



$$Importancia\ do\ Classificador = \frac{1}{2} * \log \left(\frac{1 - erro_{total}}{erro_{total}} \right)$$

*O erro total do classificador é a soma dos pesos das amostras que ele classificou incorretamente

$$Ic = \frac{1}{2} \times \log \left(\frac{1 - 1/8}{1/8} \right) = 0,97$$

Treinando um Adaboost

- A amostra que foi incorretamente classificada pelo stump precisa ter seu peso aumentado

Dor no Peito	Artérias Bloqueadas	Massa do Paciente	Doença no Coração	Peso da Amostra
Sim	Sim	93	Sim	1/8
Não	Sim	81	Sim	1/8
Sim	Não	95	Sim	1/8
Sim	Sim	75	Sim	1/8
Não	Sim	70	Não	1/8
Não	Sim	56	Não	1/8
Sim	Sim	76	Não	1/8
Sim	Sim	78	Não	1/8

$$\text{Novo peso da amostra} = \text{peso atual} \times e^{\text{importância do classificador}}$$

$$\text{Novo peso da amostra} = \left(\frac{1}{8}\right) \times e^{0,97} = 0,33$$

Treinando um Adaboost

- Depois, precisamos diminuir os pesos das outras amostras

Dor no Peito	Artérias Bloqueadas	Massa do Paciente	Doença no Coração	Peso da Amostra
Sim	Sim	93	Sim	1/8
Não	Sim	81	Sim	1/8
Sim	Não	95	Sim	1/8
Sim	Sim	75	Sim	1/8
Não	Sim	70	Não	1/8
Não	Sim	56	Não	1/8
Sim	Sim	76	Não	1/8
Sim	Sim	78	Não	1/8

Novo peso da amostra = peso atual $\times e^{-\text{importância do classificador}}$

$$\text{Novo peso da amostra} = \left(\frac{1}{8}\right) \times e^{-0,97} = 0,05$$

Treinando um Adaboost

- Mas os novos pesos, somados, não dão 100%, então precisamos normalizá-los

Dor no Peito	Artérias Bloqueadas	Massa do Paciente	Doença no Coração	Peso da Amostra	Novos Pesos	Novos Pesos Norm.
Sim	Sim	93	Sim	1/8	0,05	0,07
Não	Sim	81	Sim	1/8	0,05	0,07
Sim	Não	95	Sim	1/8	0,05	0,07
Sim	Sim	75	Sim	1/8	0,33	0,49
Não	Sim	70	Não	1/8	0,05	0,07
Não	Sim	56	Não	1/8	0,05	0,07
Sim	Sim	76	Não	1/8	0,05	0,07
Sim	Sim	78	Não	1/8	0,05	0,07

Treinando um Adaboost

- Os novos pesos podem, agora, ser usados para encontrar o próximo *stump*
- Para isso, existem duas alternativas principais:
 - Usar os pesos para calcular uma medida de impureza ponderada
 - Criar um novo dataset, sorteando as amostras do dataset original usando o peso como probabilidade de sorteio

Dor no Peito	Artérias Bloqueadas	Massa do Paciente	Doença no Coração	Pesos
Sim	Sim	93	Sim	0,07
Não	Sim	81	Sim	0,07
Sim	Não	95	Sim	0,07
Sim	Sim	75	Sim	0,49
Não	Sim	70	Não	0,07
Não	Sim	56	Não	0,07
Sim	Sim	76	Não	0,07
Sim	Sim	78	Não	0,07

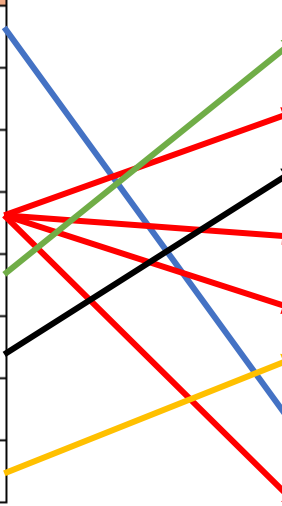
Treinando um Adaboost

- Criando o novo dataset sorteando as amostras do dataset original usando o peso como probabilidade de sorteio

Dor no Peito	Artérias Bloqueadas	Massa do Paciente	Doença no Coração
Sim	Sim	93	Sim
Não	Sim	81	Sim
Sim	Não	95	Sim
Sim	Sim	75	Sim
Não	Sim	70	Não
Não	Sim	56	Não
Sim	Sim	76	Não
Sim	Sim	78	Não

Pesos
0,07
0,07
0,07
0,49
0,07
0,07
0,07
0,07

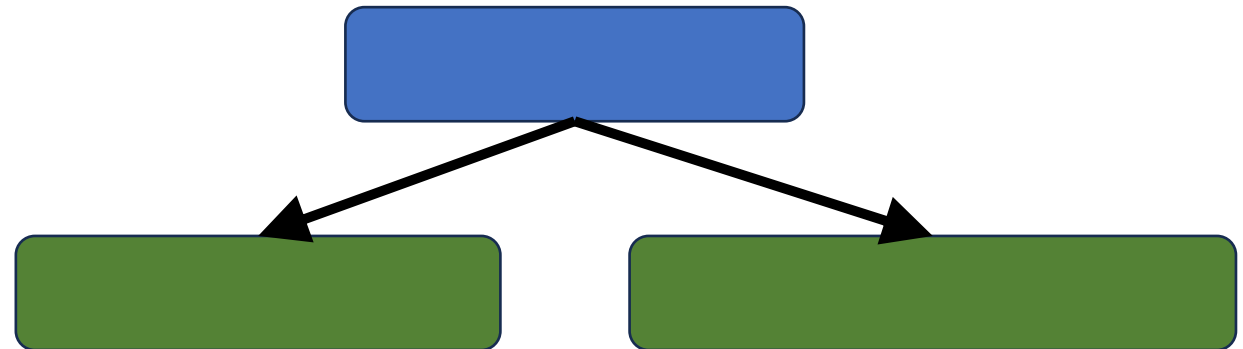
Dor no Peito	Artérias Bloqueadas	Massa do Paciente	Doença no Coração
Não	Sim	70	Não
Sim	Sim	75	Sim
Não	Sim	56	Não
Sim	Sim	75	Sim
Sim	Sim	75	Sim
Sim	Sim	78	Não
Sim	Sim	93	Sim
Sim	Sim	75	Sim



Treinando um Adaboost

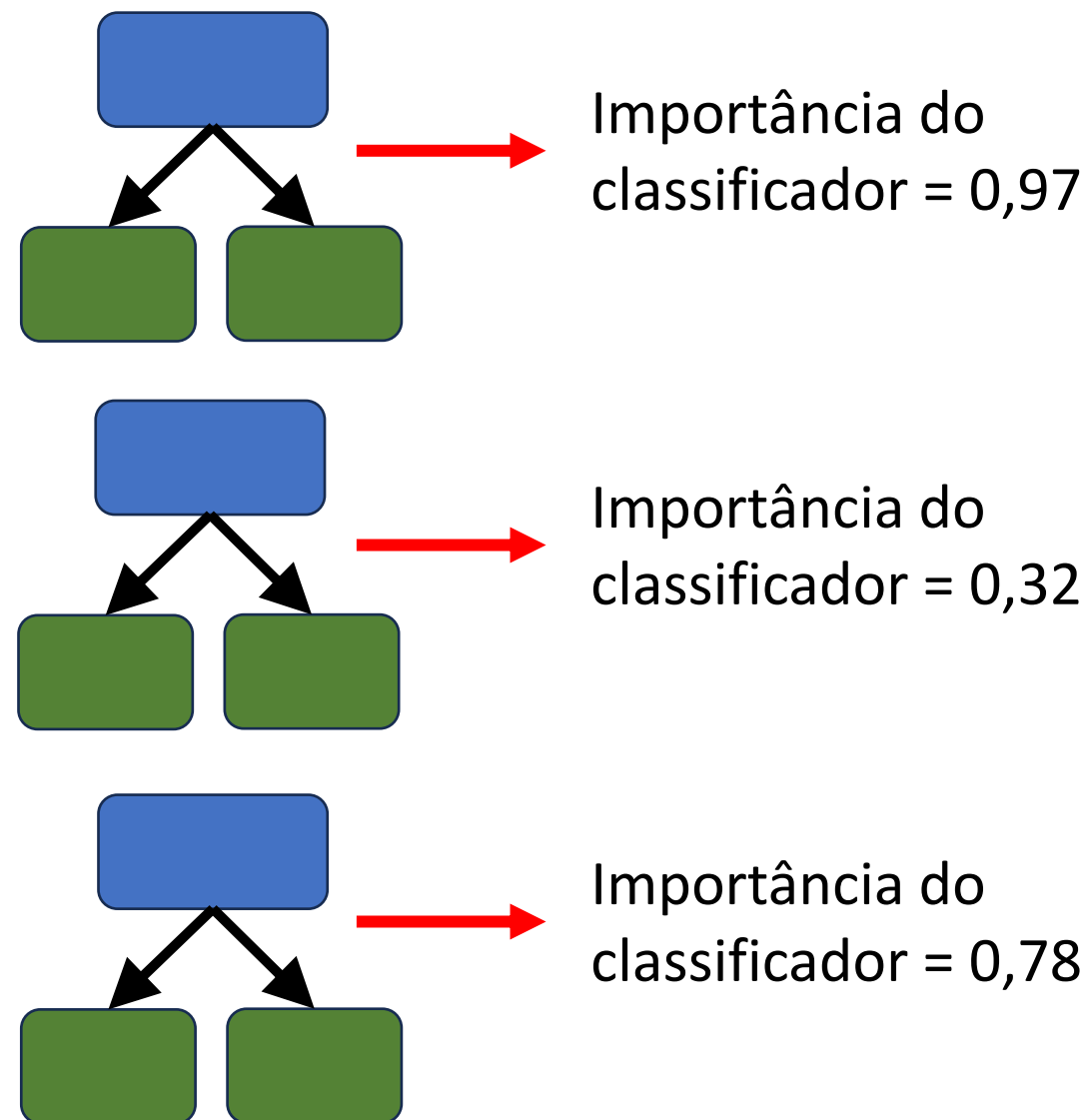
- E aí o processo se repete...
- O erro de um toco influencia na construção do próximo

Dor no Peito	Artérias Bloqueadas	Massa do Paciente	Doença no Coração	Peso da Amostra
Não	Sim	70	Não	1/8
Sim	Sim	75	Sim	1/8
Não	Sim	56	Não	1/8
Sim	Sim	75	Sim	1/8
Sim	Sim	75	Sim	1/8
Sim	Sim	78	Não	1/8
Sim	Sim	93	Sim	1/8
Sim	Sim	75	Sim	1/8



Treinando um Adaboost

- Para cada novo stump, temos a sua importância calculada
- Se a soma das importâncias dos stumps que fizeram a predição “verdade” for maior que os que predizeram “falso”, o resultado da predição do Adaboost é “verdade”
- Ou seja, fazemos a inferência em cada stump individualmente e somamos as importâncias dos classificadores. A classe com maior importância corresponde a predição final.



Gradient Boosting

- É um método de *ensemble* **sequencial** que constrói um modelo forte adicionando, a cada etapa, um novo modelo que **corrige os erros residuais do modelo anterior**.
- A correção é feita por meio da **minimização do erro (função de perda) usando gradientes**.

- Muitos aspectos do Gradient Boosting são semelhantes ao Adaboost
 - Gradient Boosting permite que as **árvores tenham número de folhas maiores** (em geral, de 8 a 32 folhas)
- Cada preditor também é construído a partir dos erros dos preditores anteriores
- Diferente do Adaboost, cada preditor é escalado por um valor fixo.
- Neste caso, o fator usado para escalar cada preditor é chamado de **learning rate**. É definido como um hiperparâmetro

- Gradient Boosting (GB) pode ser usado tanto para regressão quanto classificação
- Para Regressão:
 - A técnica começa com o cálculo da média do target

Massa média = 71,2 kg

Altura	Cor Favorita	Gênero	Massa
1,6	Azul	Masculino	88
1,6	Verde	Feminino	76
1,5	Azul	Feminino	56
1,8	Vermelho	Masculino	73
1,5	Verde	Masculino	77
1,4	Azul	Feminino	57

- Essa média é a primeira predição do GB, mas ele não para por aqui...

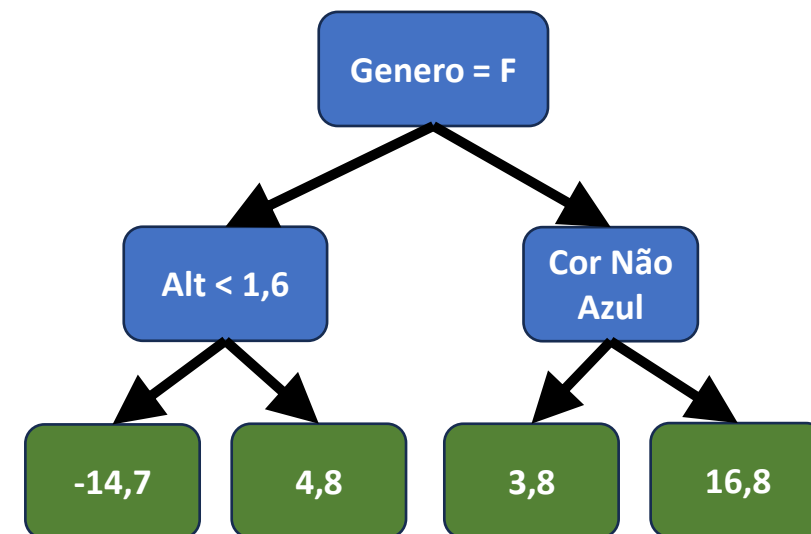
- Depois de calcular a média, verifica-se o erro da predição

Resíduo = Valor observado - média

Altura	Cor Favorita	Gênero	Massa	Resíduo
1,6	Azul	Masculino	88	16,8
1,6	Verde	Feminino	76	4,8
1,5	Azul	Feminino	56	-15,2
1,8	Vermelho	Masculino	73	1,8
1,5	Verde	Masculino	77	5,8
1,4	Azul	Feminino	57	-14,2

- A partir daí constrói-se uma árvore para prever os resíduos obtidos:

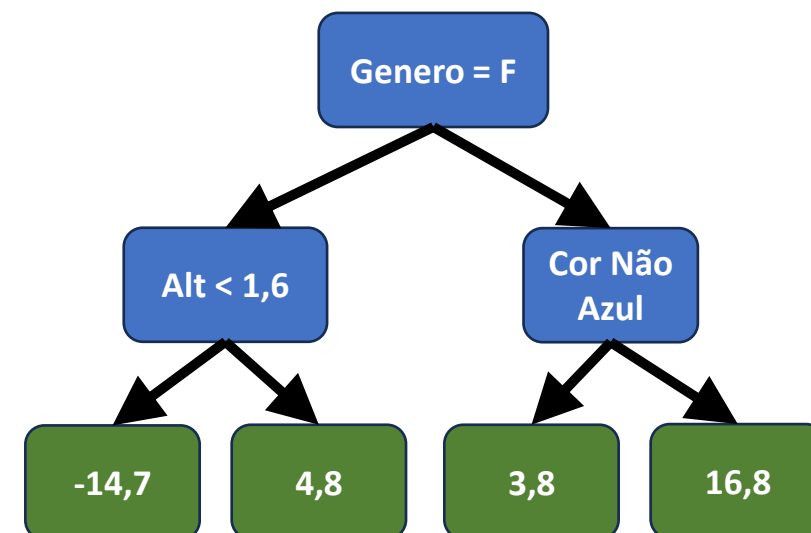
Altura	Cor Favorita	Gênero	Massa	Resíduo
1,6	Azul	Masculino	88	16,8
1,6	Verde	Feminino	76	4,8
1,5	Azul	Feminino	56	-15,2
1,8	Vermelho	Masculino	73	1,8
1,5	Verde	Masculino	77	5,8
1,4	Azul	Feminino	57	-14,2



- Com a árvore, é possível combinar seus resultados com a média anteriormente obtida e a taxa de aprendizado (learning rate):

$$\text{Valor Predito} = \text{Média} + (\text{Taxa de Aprendizado}) \times (\text{Resultado da Árvore})$$

Altura	Cor Favorita	Gênero	Massa	Resíduo
1,6	Azul	Masculino	88	16,8
1,6	Verde	Feminino	76	4,8
1,5	Azul	Feminino	56	-15,2
1,8	Vermelho	Masculino	73	1,8
1,5	Verde	Masculino	77	5,8
1,4	Azul	Feminino	57	-14,2



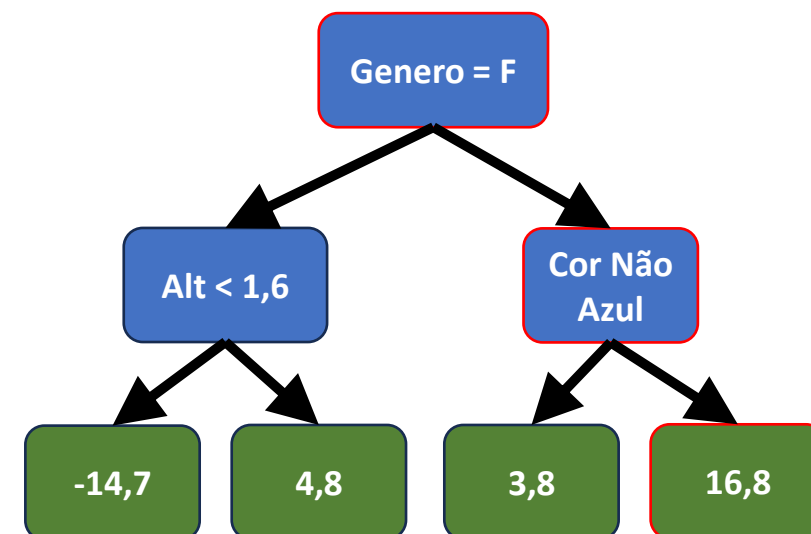
- Com a árvore, é possível combinar seus resultados com a média anteriormente obtida e a taxa de aprendizado (learning rate):

$$\text{Valor Predito} = \text{Média} + (\text{Taxa de Aprendizado}) \times (\text{Resultado da Árvore})$$

Exemplo para a primeira amostra (considere que a taxa é 0,1):

$$\text{Valor Predito} = 71,2 + 0,1 \times 16,8 = \mathbf{72,9 \text{ kg}}$$

Altura	Cor Favorita	Gênero	Massa	Resíduo
1,6	Azul	Masculino	88	16,8
1,6	Verde	Feminino	76	4,8
1,5	Azul	Feminino	56	-15,2
1,8	Vermelho	Masculino	73	1,8
1,5	Verde	Masculino	77	5,8
1,4	Azul	Feminino	57	-14,2



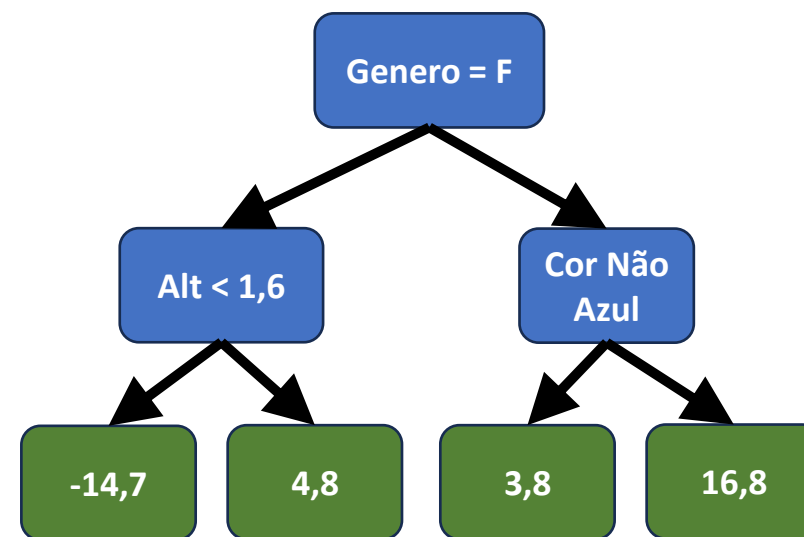
- A partir da fórmula do slide anterior, é possível atualizar os resíduos de cada amostra:

Exemplo para a primeira amostra (considere que a taxa é 0,1):

Valor Predito = $71,2 + 0,1 \times 16,8 = 72,9$ kg

Resíduo = $88 - 72,9 = 15,1$

Altura	Cor Favorita	Gênero	Massa	Resíduo
1,6	Azul	Masculino	88	15,1
1,6	Verde	Feminino	76	4,3
1,5	Azul	Feminino	56	-13,7
1,8	Vermelho	Masculino	73	1,4
1,5	Verde	Masculino	77	5,4
1,4	Azul	Feminino	57	-12,7



- O processo se repete:
 - Constrói-se uma nova árvore para prever os resíduos atualizados
 - Para atualizar a predição, os resultados da nova árvore são adicionados à fórmula:

$$\begin{aligned} & \text{Média} + \\ \text{Valor Predito} = & (\text{Taxa de Aprendizado}) \times (\text{Resultado da Árvore1}) + \\ & (\text{Taxa de Aprendizado}) \times (\text{Resultado da Árvore2}) + \dots \end{aligned}$$

- A cada iteração, um novo modelo é treinado para **ajustar os resíduos** (erros) do modelo anterior.

- Gradient Boosting para Classificação:

Na próxima aula!!!

OBRIGADO PELA ATENÇÃO!



Prof. Dr. Arthur Henrique de Andrade Melani

Departamento de Engenharia Mecatrônica e Sistemas Mecânicos

Escola Politécnica da Universidade de São Paulo

melani@usp.br